EX No:05

Date:

Model Building using Classification Algorithm for Weekly Dataset

5. This question should be answered using the Weekly data set, which is part of the ISLP package. This data is similar in nature to the Smarket data from this chapter's lab, except that it contains 1, 089 weekly returns for 21 years, from the beginning of 1990 to the end of 2010.

LOAD LIBRARY:

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS, summarize)
from ISLP import confusion_table
```

LOAD DATA:

```
weekly = load_data('Weekly')
weekly.head()
```

(a) Produce some numerical and graphical summaries of the Weekly data. Do there appear to be any patterns?

CODE:

```
print(weekly.describe(include='all'))
```

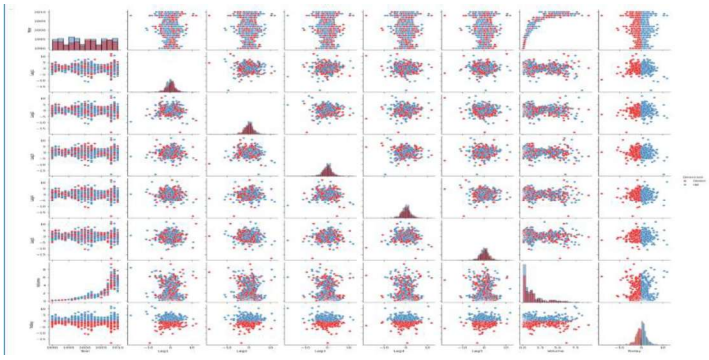
```
                Year         Lag1         Lag2         Lag3         Lag4  \
count    1089.000000  1089.000000  1089.000000  1089.000000  1089.000000
unique           NaN          NaN          NaN          NaN          NaN
top              NaN          NaN          NaN          NaN          NaN
freq             NaN          NaN          NaN          NaN          NaN
mean     2000.048669     0.150585     0.151079     0.147205     0.145818
std         6.033182     2.357013     2.357254     2.360502     2.360279
min      1990.000000   -18.195000   -18.195000   -18.195000   -18.195000
25%      1995.000000    -1.154000    -1.154000    -1.158000    -1.158000
50%      2000.000000     0.241000     0.241000     0.241000     0.238000
75%      2005.000000     1.405000     1.409000     1.409000     1.409000
max      2010.000000    12.026000    12.026000    12.026000    12.026000

                Lag5       Volume        Today Direction
count    1089.000000  1089.000000  1089.000000      1089
unique           NaN          NaN          NaN         2
top              NaN          NaN          NaN        Up
freq             NaN          NaN          NaN       605
mean        0.139893     1.574618     0.149899       NaN
std         2.361285     1.686636     2.356927       NaN
min       -18.195000     0.087465   -18.195000       NaN
25%        -1.166000     0.332022    -1.154000       NaN
50%         0.234000     1.002680     0.241000       NaN
75%         1.405000     2.053727     1.405000       NaN
max        12.026000     9.328214    12.026000       NaN
```

CODE:

```
sns.pairplot(weekly, hue='Direction', palette='Set1', diag_kind='hist')
```

OUTPUT:



(b) Use the full data set to perform a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Use the summary function to print the results. Do any of the predictors appear to be statistically significant? If so, which ones?

CODE:

```
x=MS(['Lag1','Lag2','Lag3','Lag4','Lag5','Volume']).fit_transform(weekly)
y = (weekly['Direction'] == 'Up').astype(int)
model = sm.GLM(y, x, family=sm.families.Binomial())
result = model.fit()
print(result.summary())
```

OUTPUT:

Generalized Linear Model Regression Results

Dep. Variable:	Direction	No. Observations:	985
Model:	GLM	Df Residuals:	983
Model Family:	Binomial	Df Model:	1
Link Function:	Logit	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-675.27
Date:	Thu, 15 Oct 2026	Deviance:	1350.5
Time:	21:18:43	Pearson chi2:	985.
No. Iterations:	4	Pseudo R-squ. (CS):	0.004221
Covariance Type:	nonrobust		

	coef	std err	z	P>|z|	[0.025	0.975]
const	0.2033	0.064	3.162	0.002	0.077	0.329
Lag2	0.0581	0.029	2.024	0.043	0.002	0.114

(c) Compute the confusion matrix and overall fraction of correct predictions. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

CODE:

```
pred = result.predict(x)
pred = np.array(['Down']*1089)
pred[result.predict(x) > 0.5] = 'Up'
confusion_table(weekly['Direction'], pred)
```

OUTPUT:

Truth	Down	Up
Predicted		
Down	54	430
Up	48	557

(d) Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Compute the confusion matrix and the overall fraction of correct predictions for the held out data (that is, the data from 2009 and 2010).

CODE:

```
tr = weekly['Year']< 2009
tr_data = weekly[tr]
te  data = weekly[~tr]
x = tr_data[['Lag2']]
x = sm.add_constant(x)
y = (tr_data['Direction']=='Up').astype(int)
model = sm.GLM(y, x, family=sm.families.Binomial()).fit()
print(model.summary())
te = te_data[['Lag2']]
te_m = MS(te).fit_transform(te_data)
probs = model.predict(te_m)
pred = np.array(["Down"]*len(te_data))
pred[probs > 0.5] = "Up"
confusion_table(pred, te_data.Direction)
```

OUTPUT:

	intercept	Lag2
985	1.0	-1.698
986	1.0	6.760
987	1.0	-4.448
988	1.0	-4.518

(e) Repeat (d) using LDA.

CODE:

```
X_train = tr_data[['Lag2']]
X_test = te_data[['Lag2']]
y_train = tr_data['Direction']
y_test = te_data['Direction']
lda_model = LinearDiscriminantAnalysis()
lda_model.fit(X_train, y_train)
lda_pred = lda_model.predict(X_test)
confusion_table(y_test, lda_pred)
```

OUTPUT:

Truth Predicted	Down	Up
Down	9	34
Up	5	56

```
Accuracy: 0.625
```

(f) Repeat (d) using QDA.

CODE:

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

qda  model = QuadraticDiscriminantAnalysis()
qda_model.fit(X_train, y_train)
qda_pred = qda_model.predict(X_test)
confusion_table(y_test, qda_pred)
```

OUTPUT:

Truth	Down	Up
Predicted		
Down	0	43
Up	0	61

CODE:

```
print("Accuracy:", np.mean(qda_pred == y_test))
```

OUTPUT:

```
Accuracy: 0.625
```

(g) Repeat (d) using KNN with K =1.

CODE:

```
from sklearn.neighbors import KNeighborsClassifier

knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, y_train)
knn1_pred = knn1.predict(X_test)
confusion_table(y_test, knn1_pred)
```

OUTPUT:

Truth	Down	Up
Predicted		
Down	22	21
Up	31	30

CODE:

```
print("Accuracy:", np.mean(knn1_pred == y_test))
```

OUTPUT:

```
Accuracy: 0.5
```

(h) Which of these methods appears to provide the best results on this data?

CODE:

```
results = pd.DataFrame({
   'Method': ['Logistic Regression', 'LDA', 'QDA', 'KNN (K=1)'],
   'Accuracy': [
      np.mean(pred == te_data['Direction']),
      np.mean(lda_pred == y_test),
      np.mean(qda_pred == y_test),
      np.mean(knn1_pred == y_test), ]})
results.sort_values('Accuracy', ascending=False)
```

OUTPUT:

	Method	Accuracy
0	Logistic Regression	0.625000
1	LDA	0.625000
2	QDA	0.586538
3	KNN (K=1)	0.500000

INTERPRETATION:

- (a) The Weekly data has 1,089 weekly returns from 1990 to 2010. The lag returns average about 0.15% with a standard deviation of about 2.36 (range -18.2 to 12.0); 605 weeks are Up and 484 are Down (55.6% Up). The pairplot shows the Up and Down classes overlapping almost completely on every lag variable, so no clear pattern is visible.
- (b) Lag2 is the predictor that stands out: in the summary shown its coefficient is 0.0581 (p = 0.043), significant at the 5% level, while the model's pseudo R-squared is only about 0.004, i.e. it explains very little.
- (c) On the full data the model's accuracy is (54 + 557) / 1,089 = 56.1%. It predicts Up in 987 of 1,089 weeks: it correctly labels 557 of the 605 Up weeks (92.1%) but only 54 of the 484 Down weeks (11.2%). Its main mistake is calling Down weeks Up (430 false Ups), and 56.1% is barely above the 55.6% obtained by always predicting Up. (In the record's tables the rows are the actual classes.)
- (d) Logistic regression with Lag2 (trained 1990 to 2008, tested on the 104 weeks of 2009 to 2010) reaches 62.5% accuracy.
- (e) LDA also reaches 62.5%: it predicts Up for 90 of the 104 test weeks, correctly identifying 56 of 61 Up weeks but only 9 of 43 Down weeks.
- (f) QDA reaches 58.7% because it predicts Up for every test week (61 of 104 correct), which is no better than always guessing Up.
- (g) KNN with K = 1 reaches only 50.0%, the same as a coin flip; a single neighbour fits the noise.
- (h) Logistic regression and LDA tie as best (62.5%), ahead of QDA (58.7%) and KNN (50.0%), so the simple linear boundary works best here. With only 104 test weeks, one week is about one percentage point, so these differences are small, and the lag variables carry very little predictive signal about weekly market direction.